RUNBOOK — How to Run Everything and Reproduce Every Figure

HACKSIMUL8 2026 · 6-DOF UAV Quadcopter For anyone who has to demonstrate this work, answer "*how did you get that graph?*", or re-run it from scratch.

---

0. Before you start

```
cd 'C:\Users\LENOVO\Downloads\HACKSIMU8\solution'
```

Every command below assumes you are in that folder. All scripts read their parameters from `quad_params.m` — nothing is hard-coded anywhere else, so changing a parameter there propagates through the whole project.

Required toolboxes

Toolbox	Used for
MATLAB	everything
Simulink	`quad_altitude.slx`
Control System Toolbox	`tf`, `ss`, `pidtune`, `margin`, `pzmap`
Deep Learning Toolbox	`fitnet`, `train` (Task 3)
Parallel Computing Toolbox	`parfor` in Tasks 3 and 4 (optional — falls back to serial)
Simulink Control Design	PID Tuner, `linearize`
Stateflow	installed, not currently used

Check with `smoke_test` — it reports each one.

---

1. The 30-second demo (if a judge says "show me")

```
verify_all                      % 25 checks, ~2 min, all should pass
open_system('quad_altitude')    % then press Run, double-click the Scope
```

`verify_all` prints a live pass/fail for every claim in the documentation. It **re-derives** results rather than loading saved ones, so it is genuine evidence, not a replay.

---

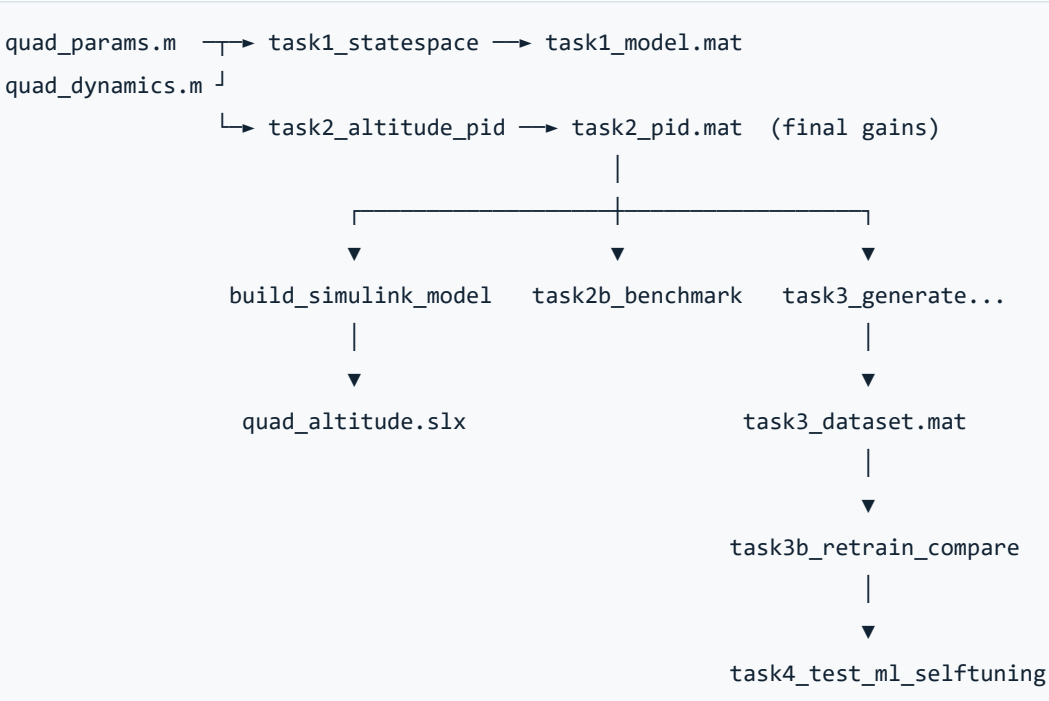
2. Full reproduction, in dependency order

Run these in sequence. Later scripts' `.mat` files.

#	Command	Time	Produces
1	`smoke_test`	~30 s	10-point sanity check
2	`task1_statespace`	~5 s	`task1_model.mat`
3	`task2_altitude_pid`	~60 s	`task2_pid.mat` ← **the final gains**
4	`task2b_zn_tyreus_luyben`	~30 s	`task2b_benchmark.mat`
5	`build_simulink_model`	~10 s	`quad_altitude.slx`
6	`task3_generate_data_train_ml`	**~13 min**	`task3_dataset.mat`, `task3_model.mat`
7	`task3b_retrain_compare`	~60 s	overwrites `task3_model.mat` with the best class
8	`task4_test_ml_selftuning`	~5-9 min	`task4_results.mat`
8b	`task4_stress_test`	~1-2 min	`task4_stress_results.mat`, figure 08
9	`export_figures`	~60 s	all PNGs in `figures/`
10	`verify_all`	~2 min	25-check verification

Or simply `run_all` for steps 2, 3, 6, 8.

Dependency chain — why order matters



If you change `alt_cost.m` or the gains in `task2_pid.mat`, everything downstream of it is stale and must be re-run. This bit us once during development.

---

3. Every figure — which script made it, and what it shows

All figures live in `solution/figures/` at 150 dpi with a white background. `export_figures.m` **regenerates all of them in one call.**

`01_task1_openloop_divergence.png`

- **Made by:** `export_figures.m` (also plotted inline by `task1_statespace.m`)
- **Shows:** altitude with a +2% thrust step at t = 1 s, no controller
- **How:** integrates `quad_dynamics` with `ode45` for 3 s at `U1 = W·(1 + 0.02·(t>1))`
- **Point:** a 2% thrust error diverges as t^2. The quadcopter cannot fly open loop.

`02_task1_four_channels_pzmap.png`

- **Made by:** `export_figures.m`
- **Shows:** pole-zero maps of altitude, roll, pitch and yaw
- **How:** builds each channel transfer function from `quad_params` and calls `pzmap`
- **Point:** every channel is a double integrator — both poles at the origin.

`03_task2_tuning_comparison.png`

- **Made by:** `export_figures.m` (reads `task2_pid.mat`)
- **Shows:** step response and thrust for all four tuning methods
- **How:** replays each method's gains through `sim_altitude` for 8 s, overlays them
- **Point:** Method D (ITAE-optimised) is fastest with zero overshoot. The lower panel shows thrust touching the 20.248 N saturation limit — realistic, not a bug.

`04_feedback_linearisation_tilt.png`

- **Made by:** `export_figures.m`
- **Shows:** altitude at 0, 0.15, 0.30 and 0.45 rad of pitch; lower panel is the same data in millimetres
- **How:** `sim_altitude` with `opt.tilt_theta = @(t) tilt*(t>3)` for each angle
- **Point:** altitude is essentially unaffected by tilt — sag stays in the micrometre range up to 25.8°.

`05_fixed_gains_degrade.png` ← **the bridge to Task 3**

- **Made by:** `export_figures.m`
- **Shows:** the same Task 2 gains at payload ×1.0, ×1.4, ×1.8, ×2.2
- **How:** `sim_altitude` with `P.m` scaled but `opt.m_ctrl` **held at the nominal mass**, so the controller does not know about the payload
- **Point:** up to 36 cm of steady-state error. This is why Task 3 exists.
- **Key detail if asked:** the `m_ctrl` option is what makes the payload genuinely unknown. Without it the controller compensates perfectly and the plot is flat — that was a real bug we found and fixed.

`06_task3_gains_vs_mass.png`

- **Made by:** `export_figures.m` (reads `task3_dataset.mat`)
- **Shows:** optimal Kp, Ki, Kd against payload for all 150 sampled conditions
- **How:** scatter of the optimisation labels `Y` against the true mass ratio `COND(:,1)`
- **Point:** Ki rises clearly with payload; Kp and Kd scatter. That visual difference *is* the R^2 story — Ki is identifiable, the others are not.

`07_reference_paper_benchmark.png`

- **Made by:** `task2b_zn_tyreus_luyben.m`
- **Shows:** upper panel — step to 2.0 m for all four designs; lower panel — the same designs under a 0.30 rad pitch step at t = 10 s
- **How:** each design is simulated in the configuration its author intended — the three from Mien & Tu as plain PID (`use_fbl = false`, matching their Eq. 27), ours feedback-linearised
- **Point:** we have the lowest ITAE of all four, and 253× less sag under tilt.

`08_task4_stress_test.png`

- **Made by:** `task4_stress_test.m`
- **Shows:** left panel — histogram of ITAE improvement across 120 randomly sampled conditions; right panel — improvement vs mass ratio, coloured by wind
- **How:** 120 conditions drawn from the full envelope with a seed disjoint from both training (`rng 42`) and the curated test cases (`rng 7`); fixed PID vs self-tuner on each, no oracle (too expensive to run 120×)
- **Point:** 90.8% of conditions improve (median +73.6%), but a real failure region exists — moderate payload combined with wind, visible as the cluster of negative-improvement points and confirmed by `corr(improvement, wind) = -0.310`.

---

4. The Simulink model

```
open_system('quad_altitude')
```

Structure, left to right:

```
Step (z_ref) → Sum(+−) → PID Controller → +g → ×m → ÷(cos ϕ·cos θ)
                ↑                                          ↓
                │                                 Saturation [0, 20.248] N
                │                                          ↓
                │                       ×(cos ϕ·cos θ) → ×(1/m) → +(−g)
                │                                          ↓
                └──────────────── ∫ ← ∫ ←──────────────────┘
                                  z
```

Block	Value	Meaning
PID	Kp=30.1847, Ki=0, Kd=10.3994	from `task2_pid.mat`
`mass` gain	0.516	m, converting desired acceleration → force
`inv_mass` gain	1.938	**1/m** — Simulink's Gain block multiplies, so dividing by m means multiplying by its reciprocal
`gravity`	−9.81	so the vehicle falls with no thrust
`g_ff`	+9.81	gravity feed-forward, added *before* the mass gain
`U1_sat`	[0, 20.248] N	the rotors' real limit
`theta_cmd`	0.3 rad step at t = 4 s	the tilt disturbance

Solver: fixed-step `ode3`, dt = 0.001, stop 12 s. **Parameters load automatically** via the model's `PreLoadFcn` callback (`P = quad_params();`).

What to point at during a demo: run it, open the Scope, and note that at t = 4 s the pitch steps to 17.2° and **altitude does not move** — thrust rises from 5.062 N to 5.30 N (= 5.062/cos 0.3) to compensate exactly.

To rebuild after changing gains: `build_simulink_model` — it reads `task2_pid.mat`, so the model can never drift out of sync with the tuning.

---

5. Reproducing the headline numbers

Claim	Command that proves it
`max|A - A_num| = 1.636e-12`	`task1_statespace` or `verify_all` section [B]
`rank(ctrb) = 12`	`task1_statespace` or `verify_all` section [D]
Final gains, Ts = 0.95 s, OS = 0.00%	`task2_altitude_pid`
Sag 0.000134 m at 17.2°	`verify_all` section [F]
Simulink sag 0.000003 m	`verify_all` section [H]
Beats all three paper designs	`task2b_zn_tyreus_luyben`
Ki rises with payload	`task3b_retrain_compare` + figure 06

---

6. Troubleshooting

"Undefined function 'pidtune'" — Control System Toolbox not on the path. Run `ver`.

`parfor` **runs serially** — Parallel Computing Toolbox missing. Everything still works, just ~4× slower. `task3_generate_data_train_ml` prints `workers: 0` when this happens.

Task 3 takes far longer than expected — it is 150 conditions × up to 180 optimiser evaluations × one 8-second simulation each ≈ 345 million derivative evaluations. Set `FAST = true` (already the default) or

reduce `N` at the top of the script.

Figures come out on a dark background — R2026a defaults to a dark theme. `export_figures` forces white; use it rather than saving from a figure window directly.

Simulink model gains do not match the docs — run `build_simulink_model` to resync from `task2_pid.mat`.

`corr` undefined — Statistics Toolbox is not installed. We use `corrcoef` from base MATLAB instead; this is deliberate.

---

7. What each MATLAB file does

Model (Task 1)

File	Role
`quad_params.m`	All Table 1 parameters + derived values. **Single source of truth.**
`quad_dynamics.m`	Nonlinear 6-DOF equations: 12 states in, $\dot{x}$ out
`rotor2U.m`	Rotor speeds → U1–U4 via $\tau = r \times F$
`task1_statespace.m`	Builds and verifies A, B, C, D

Control (Task 2)

File	Role
`task2_altitude_pid.m`	Four tuning methods; produces the final gains
`task2b_zn_tyreus_luyben.m`	Benchmark against Mien & Tu (2024)
`sim_altitude.m`	The nonlinear closed-loop simulator used by everything
`perf_metrics.m`	Tr, Ts, OS, SSE, ITAE, IAE, ISE, control effort
`alt_cost.m`	The objective Method D and all Task 3 labels minimise
`build_simulink_model.m`	Generates `quad_altitude.slx` programmatically

Machine learning (Tasks 3–4)

File	Role
`task3_generate_data_train_ml.m`	150 conditions → features + optimal-gain labels → trained model
`task3b_retrain_compare.m`	Four model classes, selected on held-out data
`task3c_relabel_warmstart.m`	Re-labels after a cost change, warm-started
`task4_test_ml_selftuning.m`	Two-phase self-tuner, 5 test cases, three-way comparison

Validation and output

File	Role
`smoke_test.m`	10-point check — run after any edit
`verify_all.m`	25 independent checks — re-derives rather than replays
`export_figures.m`	All figures as white-background 150 dpi PNGs
`run_all.m`	Tasks 1–4 in order

---

8. The Task 4 adaptation rule, and why it looks the way it does

`task4_test_ml_selftuning.m` and `task4_stress_test.m` both apply the SAME rule to the model's raw prediction before using it - keep these two files in sync if either changes:

```
g_ml = PID_base;
g_ml(1) = min(max(g_raw(1), PID_base(1)), Y_hi(1));   % Kp: may rise, never fall
g_ml(2) = min(max(g_raw(2), Y_lo(2)),      Y_hi(2));   % Ki: free within training range
% Kd is left at PID_base - never adapted
```

`Y_lo`, `Y_hi` are the min/max of `task3_dataset.mat`'s `Y` (the training labels) - the model is allowed to interpolate within what it was trained on, never to extrapolate beyond it. This asymmetric rule was arrived at empirically (three attempts, measured against each other - see `TASK4_final.md` section 2), not assumed in advance. If the model or dataset changes, re-derive `Y_lo` / `Y_hi` from the new `task3_dataset.mat` - do not hardcode them.

9. Key options in `sim_altitude.m`

This one function powers every simulation, so it is worth knowing its options:

Option	Default	Effect
`use_fbl`	`true`	feedback linearisation on/off. **Set false to reproduce the reference paper's plain-PID configuration.**
`m_ctrl`	`P.m`	the mass the *controller assumes*. Set to nominal while `P.m` holds the true mass to model an **unknown payload**
`tilt_phi`, `tilt_theta`	`@(t) 0`	roll/pitch profiles, as function handles
`dist`	`@(t) 0`	vertical disturbance force in N (wind, gusts)
`noise_std`	`0`	altitude measurement noise
`z0`, `zdot0`, `I0`, `t0`	`0`	initial conditions, including integrator state for bumpless gain switching
`nsub`	`10`	RK4 substeps per control period. **4 is what Tasks 3 and 4 use** — evaluation fidelity must match training fidelity

Example — heavy payload the controller does not know about, with wind:

```
P = quad_params();
P.m = P.m * 1.8;  P.W = P.m * P.g;          % true mass
opt = struct('m_ctrl', 0.516, ...            % controller still assumes nominal
             'dist', @(t) 0.5, ...           % 0.5 N steady wind
             'tilt_theta', @(t) 0.2*(t>1.5));
R = sim_altitude([30.1847 0 10.3994], P, 1.0, 8, opt);
M = perf_metrics(R.t, R.z, 1.0, R.U1);
fprintf('settles at %.4f m, SSE %.4f m\n', R.z(end), M.SSE);
```